

Tema 1: La capa de acceso a datos con Spring y JPA

1. Spring Boot y Configuración Inicial

Spring Boot facilita la configuración del entorno mediante la auto-configuración. En nuestro proyecto, las propiedades de conexión a la base de datos y la configuración de JPA/Hibernate se centralizan en el fichero `application.yml`.

Un aspecto vital es la estrategia de generación de esquemas. En un entorno de producción, la propiedad `spring.jpa.hibernate.ddl-auto` suele establecerse en `none` para evitar que Hibernate modifique automáticamente las tablas de la base de datos, delegando esta tarea a scripts SQL explícitos.

2. Diagrama de Entidades

Antes de codificar, debemos comprender el dominio. Nuestro proyecto de reserva de entradas de cine se modela principalmente a través de las siguientes entidades:

- **User:** Representa a los usuarios (espectadores y taquilleros).
 - **Movie:** Representa una película en el catálogo.
 - **Saloon:** Define las salas de cine y su capacidad.
 - **Session:** Es la entidad central que vincula una `Movie` con un `Saloon` en una fecha y hora determinadas (`sessionStartTime`), estableciendo además un precio y el número de asientos libres.
 - **Purchase:** Representa la compra de entradas, enlazando a un `User` con una `Session`.
-

3. Modelado Básico de Entidades

Una entidad en JPA es una clase Java simple anotada con `@Entity` que se mapea directamente a una tabla en la base de datos.

La clave primaria se define con la anotación `@Id`. Es altamente recomendable delegar la generación del identificador a la base de datos utilizando `@GeneratedValue(strategy = GenerationType.IDENTITY)`.

Ejemplo fundamental:

```

@Entity
public class Movie {
    private Long id;
    private String title;

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    public Long getId() {
        return id;
    }
    // ... otros getters y setters
}

```

4. Modelado de Relaciones

4.1. Entidad Propietaria vs Entidad Inversa

La **entidad propietaria** de la relación es aquella cuya tabla en la base de datos contiene la clave foránea.

- **Regla de Examen:** Se debe utilizar `@JoinColumn` **exclusivamente** en la entidad propietaria.
- Si la relación es bidireccional (por ejemplo, `User` y `Account`), el lado inverso (la entidad que no tiene la clave foránea, usualmente la que contiene la colección `Set<Account>`) debe utilizar el atributo `mappedBy` para indicar quién es el propietario, y **nunca** debe llevar la anotación `@JoinColumn`.

En nuestro proyecto, la entidad `Purchase` es propietaria de su relación con `Session` y con `User` utilizando `@ManyToOne` y definiendo `@JoinColumn`:

```

@ManyToOne(optional = false, fetch = FetchType.LAZY)
@JoinColumn(name = "session_id")
public Session getSession() {
    return session;
}

```

4.2. Estrategias de Recuperación (Fetch Strategies)

Siempre debemos preferir `FetchType.LAZY` para las relaciones y colecciones, cargando los datos asociados únicamente cuando se accede a ellos.

- **Regla de Examen:** Invocar el método que recupera el identificador (ej. `item.getId()`) sobre una entidad o colección cargada de forma diferida (proxy) **no** dispara una consulta a la base de datos, ya que Hibernate posee dicho valor (es la clave foránea). Acceder a cualquier otra propiedad **sí** generará una consulta a la base de datos.
-

5. Métodos de Negocio en Entidades

Aunque la mayoría del código en una entidad consiste en métodos getter y setter, podemos incluir lógica o anotaciones que resuelvan problemas de negocio a nivel de persistencia. Un ejemplo claro es el **Control de Concurrency Optimista**.

Para evitar que dos usuarios compren la última entrada de una `Session` simultáneamente, se añade un campo numérico o de marca de tiempo anotado con `@Version`. JPA incrementará este valor automáticamente en cada actualización, lanzando una excepción si detecta que otra transacción ha modificado la fila.

```
@Version
public Long getVersion() {
    return version;
}
```

6. Desarrollo de DAOs (Data Access Objects)

Spring Data JPA simplifica enormemente el desarrollo de los DAOs mediante interfaces. Al heredar de `CrudRepository` o `JpaRepository`, obtenemos métodos de persistencia básicos (`save`, `findById`, `delete`) sin necesidad de implementarlos manualmente.

6.1. Métodos derivados por nombre

Spring permite definir consultas analizando el nombre del método en la interfaz. Por ejemplo, en `UserDao`, el método `existsByUsername(String userName)` ejecutará automáticamente una consulta que verifica la existencia de un registro por el campo `userName`.

6.2. Anotación `@Query`

Para consultas complejas que no pueden (o no deben) derivarse del nombre, empleamos JPQL con la anotación `@Query`. En `SessionDao`, necesitamos encontrar las sesiones asociadas a una fecha específica ignorando la hora, para lo cual construimos explícitamente la consulta:

```
public interface SessionDao extends JpaRepository<Session, Long> {  
    @Query("SELECT s FROM Session s WHERE CAST(s.sessionStartTime AS date) =  
:date")  
    List<Session> findByDate(@Param("date") LocalDate date);  
}
```